BoardGameStatTracker

Description

An app to enable quicker and easier stat tracking for games that do not inherently track your stats for you (especially Board Games). It is intended to act as a service for board game publishers to have their designers use. It will make data analytics much easier to examine and minimize the risk taken in publishing a new board game.

Section 1:

Introduction: Board Game publishers often require their designers to track the stats for 100-150+ sessions of the game they are designing. There is no great way for them to do this, often using Excel or Sheets to track the stats. This isn't good for the designer, because it is a pain to record the stats. It is also a pain for the publishers to analyze those stats due to inconsistencies and human error. This app will solve both of those issues, making the whole process simpler, cleaner, and more accurate.

Section 2:

2a: Must Have Requirements

- Completely Usable App for users
- Login Capabilities
- Functionality to track and store stats in cloud database
- Updated MySQL Database following first 3 Normal Forms
- Python API with all endpoints created
- Plan and design for Elixir WebSocket Implementation

2b: Stretch Requirements

- Implemented Elixir WebSocket
- Fleshed Out Publisher Views
- Website Implementation for viewing information (no ability to track stats)
- Publisher Specific Website Mockup (to use for selling product to publishers)

Section 3: Overview of the Product.

Workflow:

The User will login into the app like any other app. They can then move to a stats view screen, a follows feed (to see other users' tracked stats), or a screen to join or create a new session for tracking new stats. When in the screen to track stats, they will be able to switch between the the different entities to track stats as they wish (between the table and individual players). When done tracking stats, they can click a button which takes them to a screen where they can select all players who won (one, multiple, or all). Once selection has been made, all related information is saved to the database and workflow starts over (not including login).

Resources:

AAlang-GAB - https://github.com/yenrab/AALang-Gab - Used for AI Agent creation to help throughout the entire project process
Python - Language - Used for REST API
FastAPI - https://pypi.org/project/fastapi/ - Used for structure of API
Pydantic - https://pypi.org/project/pydantic/ - Used to help FastAPI with data validation and settings management
Elixir - Language - Used for session WebSocket to connect multiple users into one session
OTP - https://elixirschool.com/en/lessons/advanced/otp_concurrency (Built into Elixir) - Used for tool to simplify the WebSocket setup and readability
MySQL - Language - Used for Database
Kotlin - Language - Used for App
Probably a couple more I haven't realized I will use yet.

Data at Rest:

It will all be stored in a MySQL Database.

Data on the Wire:

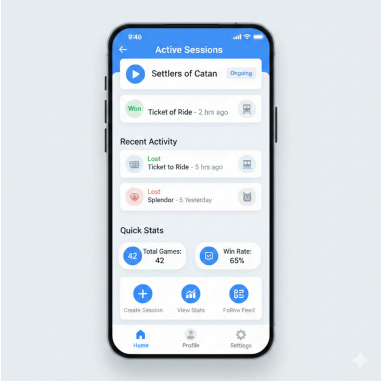
When a user is in a session, they are connected to the Elixir WebSocket. The WebSocket will keep track of all the stats being tracked for a session, periodically saving the stats to the Database. At the end of a session, all stats and an end time will be saved for the session. All parts of transmission will be necrypted eventually but may or may not be by the end of the semester. Login info will also have auth tokens and refresh tokens for users will also be maintained as well.

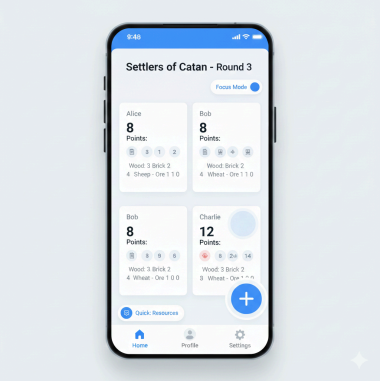
Data State:

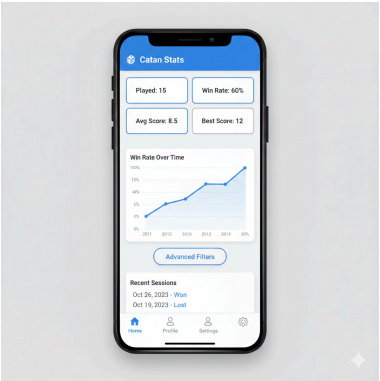
The system operates on a dual-flow architecture: **Real-time state** managed by Elixir and **Persistent state** managed by MySQL.

- **Initialization State:** The Kotlin app requests a session via the Python REST API. The API validates the user and triggers the Elixir WebSocket to spin up a dedicated process (GenServer) for that Session ID.
- **Active Tracking State (In-Flight):** Data flows through the WebSocket. As users update stats, the Elixir OTP process maintains the "source of truth" in memory. This minimizes database overhead and allows multiple users to see updates simultaneously.
- **Persistence State:** Periodically (heartbeats) and upon "Session End," the Elixir process pushes the final payload to the Python API, which validates the data using Pydantic and commits it to the MySQL database.
- **Archival State:** Once committed, the data moves to the "Stats View" state, where it is available for read-only analysis by designers and publishers via standard REST GET requests.

HMI/HCI/GUI:







Section 4: Verification

Demo:

I will demonstrate the application's viability through a **Live Board Game Session** simulation:

- **Scenario:** Two users (on separate devices) will join a single session for a fictional game.
- **Data:** We will input a pre-defined set of 10 turns of data (e.g., resource fluctuations and point gains).
- **Verification:** 1. Show real-time syncing between the two devices via WebSockets. 2. End the session and mark a winner. 3. Open the MySQL Workbench/Terminal to show the record successfully persisted in 3NF format. 4. Navigate to the "Stats View" in the app to show the historical data is now retrievable.

Testing:

Requirement	Verification Method	Acceptance Criteria	Failure Condition
Login/Auth	Attempt login with valid and invalid credentials.	JWT token is issued; user directed to Dashboard.	Access granted without token; password stored in plain text.
3NF MySQL DB	Database schema inspection via SQL DESCRIBE commands.	No transitive dependencies; no repeating groups; primary keys defined.	Redundant data across tables; manual data entry required for IDs.
WebSocket Sync	Update a stat on Device A and observe Device B.	Update reflects on Device B in <200ms.	Stat only updates after a manual page refresh.
Python API	Execute Pydantic validation tests on POST endpoints.	API returns 201 Created for valid JSON and 422 Unprocessable Entity for malformed stats.	Database accepts negative game durations or null player names.
Cloud Storage	Close the app during a session and restart.	WebSocket re-establishes state from the last saved "In-Flight" data.	All session progress is lost upon app closure.